

A dark blue vertical bar on the left side of the page. A blue arrow points to the right from the bar, containing the date.

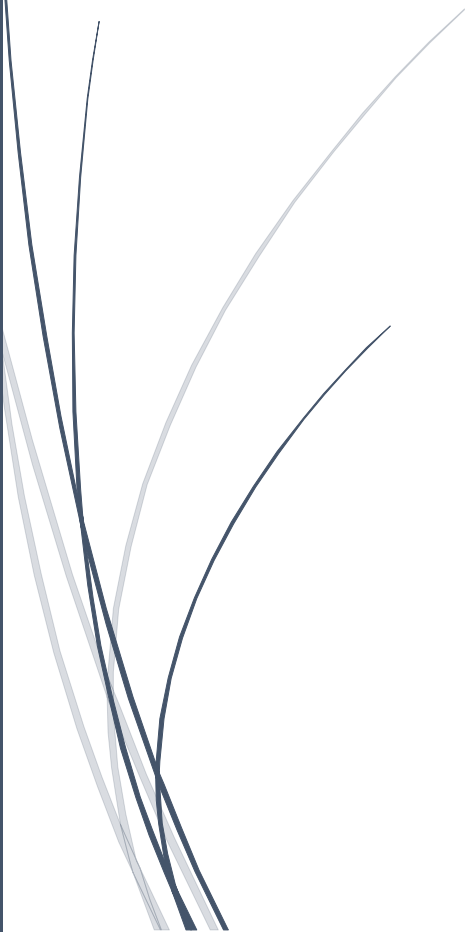
5/1/2026

IoT Based Smart Irrigation System

Mechatronics (PD - 215)

Parthib Kumar Dey (Sr No 26787)

Abha Singh Sardar (Sr No)

Several thin, curved lines in shades of blue and grey, resembling stylized grass or reeds, located in the bottom left corner of the page.

IoT Based Smart Irrigation System

Introduction : Water scarcity and inefficient irrigation practices have long been a challenge in agriculture and everyday plant care. Traditional watering methods rely on fixed schedules or manual effort, often resulting in overwatering or crop damage. An IoT-based Smart Watering System solves this by continuously monitoring environmental conditions and making autonomous decisions about when to irrigate.

This project implements a smart irrigation system using the Raspberry Pi Pico W, programmed in MicroPython. It uses five sensors — soil moisture, water level, temperature & humidity, rain detection, and water flow — to monitor real-time conditions and automatically control a 12V pump via a relay. A 20×4 LCD provides on-site feedback, while a WiFi dashboard enables live monitoring and remote pump control through a simple REST API.

The outcome is a low-cost, efficient, and reliable system that eliminates guesswork from watering, conserves water, and demonstrates the practical power of IoT in everyday applications.

2. System Overview & Architecture

The Smart Irrigation System is designed around a central microcontroller that continuously collects environmental data, processes it through decision logic, and triggers appropriate outputs — all while remaining accessible over a local WiFi network. The system can be broadly divided into three layers: inputs, processing, and outputs.

2.1 Input Layer (Sensors)

Five sensors feed real-time data into the system, each connected to a dedicated GPIO pin on the Pico W. The capacitive soil moisture sensor (GP26) measures the water content of the soil by detecting changes in electrical capacitance. The resistive water level sensor (GP27) monitors how much water remains in the supply tank. The DHT11 sensor (GP15) captures ambient temperature and humidity, providing environmental context. The YL-83 rain detector (GP14) signals whether it is currently raining, allowing the system to avoid unnecessary watering. Finally, the YF-S401 flow sensor (GP13) tracks the actual volume of water dispensed by counting pulses generated by a spinning impeller.

2.2 Processing Layer (Raspberry Pi Pico W)

The Pico W serves as the brain of the system. Running MicroPython, it performs five key functions. First, it applies a 10-sample trimmed ADC filter to all analog sensor readings, discarding outliers and averaging the middle values to eliminate electrical noise. Second, it runs pump hysteresis logic — turning the pump on when soil moisture drops below 40%, holding between 40–60%, and turning it off above 60% — to prevent rapid on/off switching. Third, it manages a WiFi HTTP server that serves a live dashboard and accepts remote pump control commands. Fourth, it drives a two-page LCD display manager that alternates between sensor readings and network/flow data. Fifth, it performs sensor fault detection, forcing the pump off whenever a sensor returns an error value to ensure safe operation.

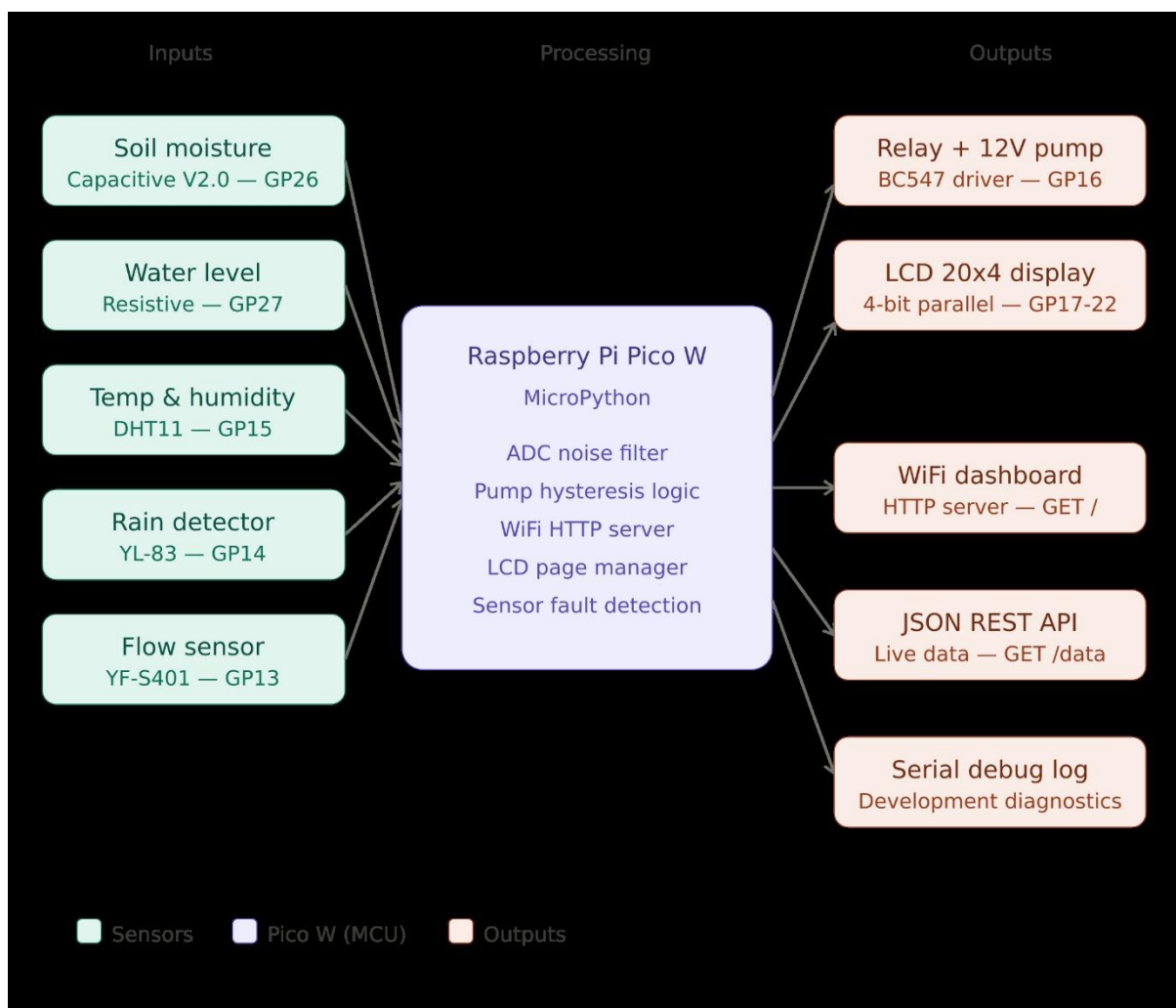
2.3 Output Layer

The system produces five types of output. The primary output is the 12V DC water pump, controlled via a relay module driven by a BC547 transistor — which bridges the Pico's 3.3V logic to the relay's 5V coil. The LCD 20×4 display provides on-site feedback across two rotating screens. The WiFi dashboard delivers a browser-accessible interface for remote monitoring. The

JSON REST API at the /data endpoint exposes all sensor values in a machine-readable format for potential integration with other systems. A serial debug log is also available during development for real-time diagnostics.

2.4 Data Flow Summary

At every main loop cycle, the Pico W reads all five sensors, filters the ADC values, evaluates pump logic, refreshes the LCD, and checks for incoming WiFi requests — all in a non-blocking sequence. A WiFi reconnect watchdog runs every 30 loops to maintain connectivity. This tight loop ensures the system responds promptly to changing conditions while remaining stable and resource-efficient on the Pico W's constrained hardware.



3. Hardware Used:

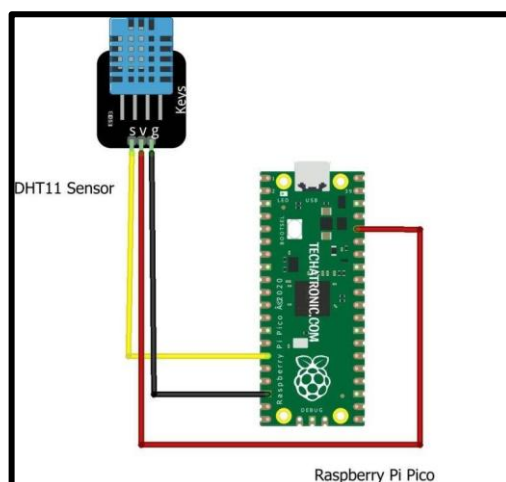
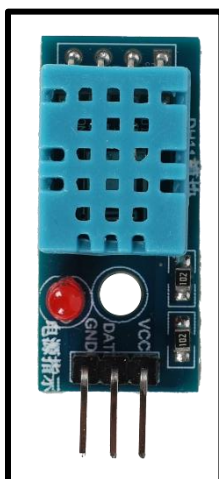
3.1. DHT11 Temperature and Humidity Sensor

3.1.1. Introduction: The DHT11 is a low-cost digital temperature and humidity sensor that integrates a capacitive humidity sensing element and an NTC thermistor for temperature measurement. It includes an internal 8-bit microcontroller that performs signal processing and provides calibrated digital output. The sensor uses a single-wire communication protocol, making it easy to interface with microcontrollers such as the Raspberry Pi Pico W. Due to its simplicity, reliability, and low power consumption, the DHT11 is widely used in environmental monitoring applications.

3.1.2. Working Principle: The DHT11 sensor operates using two sensing elements:

- Humidity sensing element (capacitive type):
The sensor measures relative humidity by detecting changes in capacitance caused by moisture in the air. As humidity increases, the dielectric constant of the sensing material changes, resulting in a measurable variation in capacitance.
- Temperature sensing element (NTC thermistor):
The thermistor changes its resistance with temperature. This resistance variation is converted into a digital value by the internal circuitry.

The internal microcontroller processes both signals, applies calibration coefficients stored in memory, and converts them into digital data. The output is transmitted using a single-wire serial communication protocol in the form of a 40-bit data frame consisting of humidity, temperature, and checksum values.



3.1.3. Communication Protocol

The DHT11 uses a proprietary single-wire communication method:

1. The microcontroller sends a start signal by pulling the data line LOW for at least 18 ms.
2. The sensor responds with an 80 μ s LOW signal followed by an 80 μ s HIGH signal.
3. The sensor then transmits 40 bits of data:
 - 8-bit integer humidity
 - 8-bit decimal humidity
 - 8-bit integer temperature
 - 8-bit decimal temperature
 - 8-bit checksum

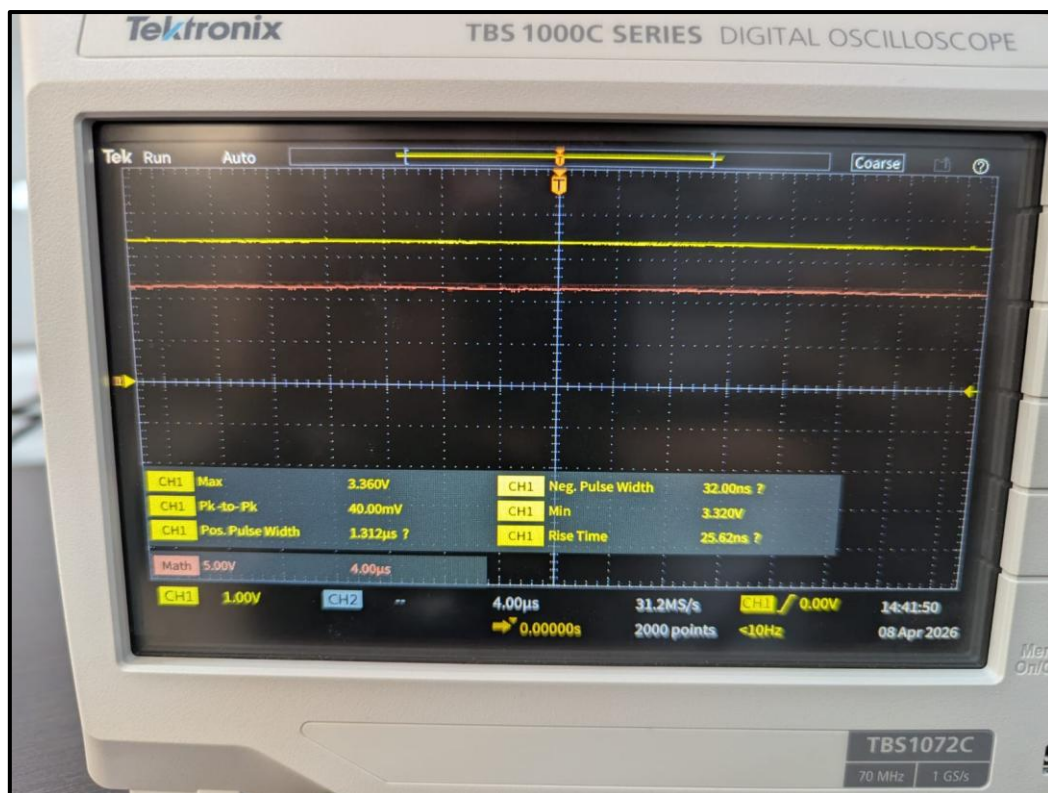
Each data bit is represented by pulse width:

- Logic '0' $\rightarrow$ short HIGH pulse ($\sim 26\text{--}28\ \mu\text{s}$)
- Logic '1' $\rightarrow$ long HIGH pulse ($\sim 70\ \mu\text{s}$)

3.1.4. Technical Specifications

Parameter	Value
Operating Voltage	3.0 – 5.5 V
Temperature Range	0°C to 50°C
Humidity Range	20% to 90% RH
Temperature Accuracy	$\pm 2^\circ\text{C}$
Humidity Accuracy	$\pm 5\%$ RH
Resolution	1°C, 1% RH
Output	Digital (single-wire)
Sampling Period	≥ 1 second
Current Consumption	0.5–2.5 mA

3.1.5. Oscilloscope Analysis: The DHT11 output signal was analyzed using a digital oscilloscope. The observed waveform shows a stable HIGH voltage level of approximately 3.3 V during idle conditions, with a peak-to-peak noise of around 40 mV, indicating a clean and stable signal. The captured waveform also shows pulse-width variations corresponding to digital data transmission. The rise time (~ 25 ns) and absence of distortion confirm good signal integrity and proper interfacing with the microcontroller. The variation in pulse widths validates the pulse-width encoding mechanism used by the DHT11 for transmitting binary data.



3.1.6. Implementation in the System

The DHT11 sensor was interfaced with the Raspberry Pi Pico W using a single GPIO pin (GP15). A pull-up resistor was used to maintain the data line at a HIGH state when idle. The microcontroller periodically initiates communication with the sensor and reads temperature and humidity values at intervals greater than one second, as required by the sensor specifications.

The measured values are:

- Displayed on the LCD interface & Sent to the web dashboard via WiFi

The DHT11 sensor is used for environmental monitoring and does not directly control the irrigation system but provides useful ambient condition data.

3.2 Water Level Sensor (Resistive Type):

3.2.1. Introduction

The water level sensor used in this system is a resistive-type analog sensor designed to detect the presence and level of water. It consists of a series of exposed parallel conductive traces printed on a PCB. The sensor operates by measuring changes in electrical conductivity when water comes into contact with these traces. Due to its simple structure, low cost, and ease of interfacing, it is widely used in water level detection applications.

3.2.2. Working Principle

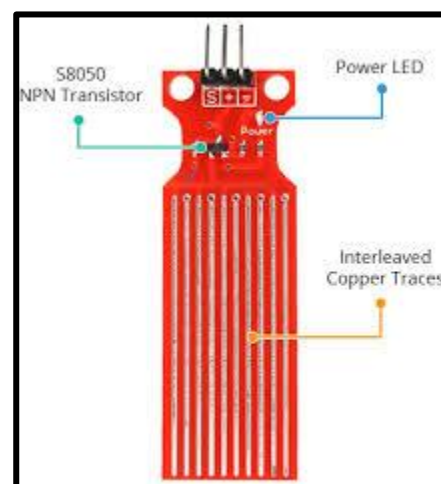
The sensor operates on the principle of electrical conductivity of water.

- The PCB contains multiple parallel exposed copper traces
- When water touches the sensor:
 - It creates a conductive path between traces
 - Electrical resistance decreases
- As water level increases:
 - More traces are submerged
 - Conductivity increases further
 - Output voltage increases

Thus, the sensor converts water level into an analog voltage signal proportional to the amount of water present.

3.2.3 Technical Specification

Parameter	Value
Operating Voltage	5V DC
Working Current	< 20 mA
Sensor Type	Analog (resistive)
Detection Area	40 mm × 16 mm
Output	Analog voltage
Sensitivity	High
Weight	~3 g



3.2.4. Oscilloscope Analysis

The output of the water level sensor was analyzed at different immersion levels using a digital oscilloscope.

Case 1: 100% Submerged

- Mean Voltage ≈ 2.03 V
- Max Voltage ≈ 2.06 V
- Peak-to-Peak Noise ≈ 40 mV

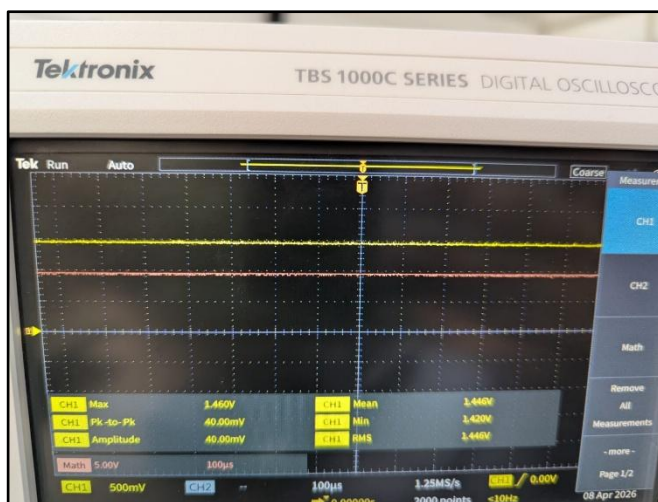
Case 2: 50% Submerged

- Mean Voltage ≈ 1.44 V
- Max Voltage ≈ 1.46 V
- Peak-to-Peak Noise ≈ 40 mV

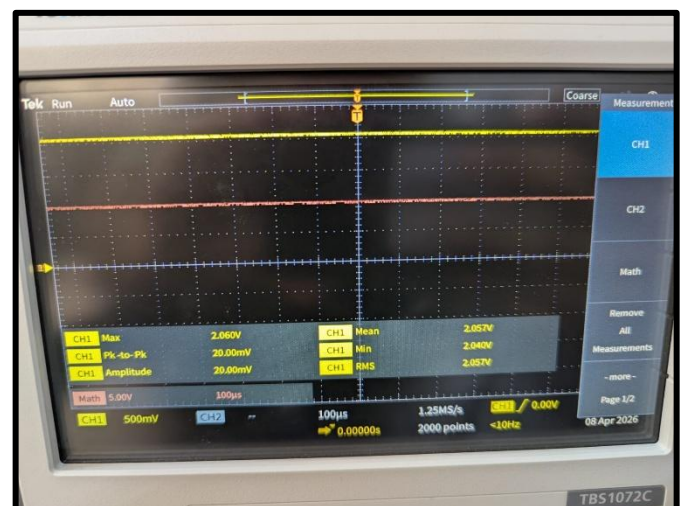
Interpretation

- Voltage increases with water level:
 - 50% $\rightarrow \sim 1.44$ V
 - 100% $\rightarrow \sim 2.03$ V
- Output is analog and proportional
- Noise level is very low (~ 40 mV), indicating stable readings

This confirms: Proper sensor functioning and reliable analog output



Oscilloscope Reading at 50% Submergence



Oscilloscope Reading at 100% Submergence

3.2.5. Calibration and Characterization

Due to the non-linear nature of the resistive sensor, calibration was performed using experimentally measured ADC values.

Condition	Raw ADC Value	Water Level (%)
Dry (no water)	~1150	0%
30% Submerged	~19800	67%
Fully Submerged	~29000	100%
Noise / Disconnected	< 800	Error

Conversion Formula

The water level percentage is calculated using:

$$\text{water\%} = (\text{raw} - 1150) \div (29000 - 1150) \times 100$$

This linear approximation is applied after calibration to convert ADC values into meaningful water level percentages.

Fault Detection Logic

- Raw < 800 → Sensor error (disconnected or fault)
- Raw < 6720 → LOW water level
- Raw > 60% → OK condition

Pump protection uses raw threshold instead of percentage for reliability

3.2.6. Implementation in the System

The water level sensor is interfaced with the Raspberry Pi Pico W via ADC channel GP27.

Processing Steps:

1. Analog signal is read using ADC
2. 10-sample trimmed mean filter is applied to reduce noise
3. Raw values are converted to percentage using calibration formula
4. Status (LOW / MED / OK / ERR) is determined

System Usage:

- Displays tank level on LCD
- Sends data to WiFi dashboard
- Used for pump safety logic

Pump Protection Logic:

- If water level is LOW → Pump OFF
- Uses raw threshold (~6720) for reliability
- Prevents dry running of pump

Real-World Sensor Selection for Large-Scale Tanks

Recommended Sensor: Ultrasonic Level Sensor

For large-scale water tanks, the most suitable sensor is an ultrasonic level sensor, which measures water level without direct contact with the liquid.

Working Principle

- The sensor is mounted at the top of the tank
- It emits ultrasonic sound waves
- Waves travel to the water surface and reflect back
- The sensor measures time of flight (ToF) of the echo

The distance is calculated using:

$$\text{Distance} = \frac{\text{Speed of sound} \times \text{Time}}{2}$$

Water level is then obtained by:

$$\text{Level} = \text{Tank height} - \text{Measured distance}$$

3.3. Rainfall Sensor (YL-83 Type)

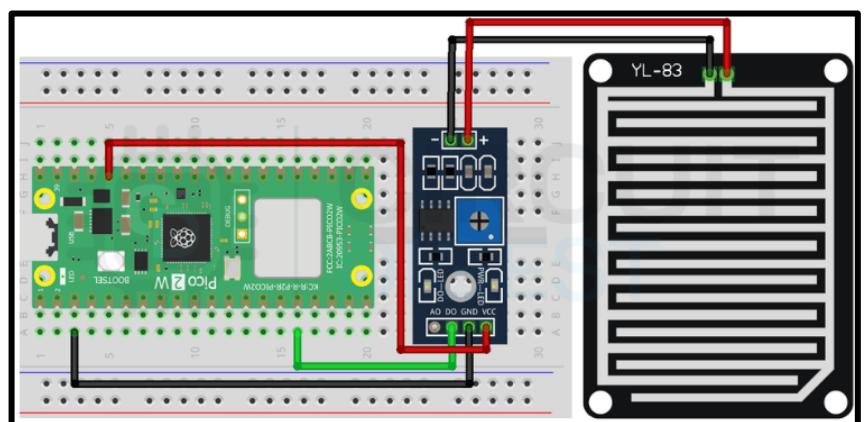
3.3.1. Introduction

The rainfall sensor used in this system is a resistive-type precipitation detection module that detects the presence of water droplets on its sensing surface. It consists of a PCB with exposed conductive traces and a comparator module that converts the analog signal into a digital output. The sensor is capable of detecting rain conditions and is widely used in environmental monitoring and automation systems.

3.3.2. Working Principle

The rainfall sensor operates based on surface conductivity and droplet detection.

- The sensing plate contains parallel exposed conductive tracks
- In dry condition:
 - No conduction between traces
 - Output voltage remains HIGH
- When rain droplets fall:
 - Water bridges the traces
 - Conductivity increases
 - Resistance decreases
 - Output voltage drops



3.3.3. Technical Specification:

Parameter	Value
Sensor Type	Resistive / Droplet detection
Output Type	Analog + Digital
Digital Output	Active LOW (rain detected)
Analog Output	~1–3 V (wet to dry)
Supply Voltage	3.3V / 5V
Detection Method	Conductivity change
Response	ON/OFF rain detection
Additional Feature	Delay circuit for rain detection

3.3.4. Oscilloscope Analysis

The sensor output was analyzed under two conditions:

Case 1: No Rain (Dry Condition)

- Mean Voltage ≈ 3.34 V
- Max Voltage ≈ 3.36 V
- Noise ≈ 40 mV

Interpretation:

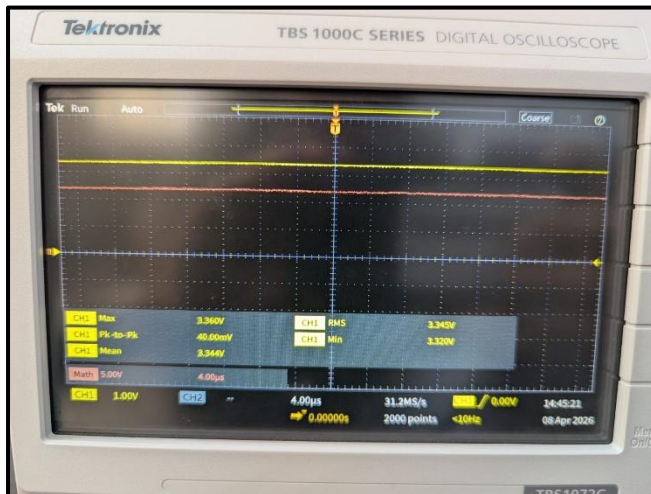
- Stable HIGH signal
- No conduction across traces
- Sensor correctly indicates no rain

Case 2: Rain Condition (Wet Surface)

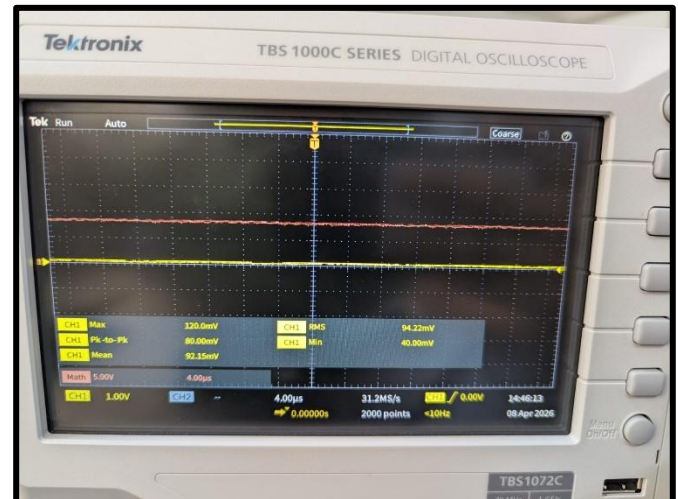
- Mean Voltage $\approx \sim 92$ mV
- Max Voltage $\approx \sim 120$ mV
- Increased variation (~ 80 mV peak-to-peak)

Interpretation:

- Significant voltage drop
- Conductive path formed by water droplets
- Sensor correctly detects rain condition



Rainfall Sensor when there is no rain: 3.344 V



Rainfall Sensor when there is rain: 92.15 mV

3.3.5 Digital Operation (Used in System)

Although the sensor provides analog output, it was used in digital mode in this system.

Condition	Output Voltage	Digital State	Meaning
Dry	~3.3 V	HIGH	No rain
Wet	~0.1 V	LOW	Rain detected

3.3.6. Implementation in the System

The rain sensor was connected to the Raspberry Pi Pico W using a digital GPIO pin (GP14).

System Behavior:

- Sensor output read as digital input
- If rain detected → system overrides irrigation

Control Logic: **If rain == TRUE → Pump OFF**

Purpose in System:

- Prevent unnecessary irrigation
- Conserve water
- Protect crops from overwatering

3.4. Water Flow Sensor (YF-S401)

3.4.1. Introduction

The YF-S401 water flow sensor is used to measure the flow rate of water in the irrigation system. It is installed inline with the pipe and generates digital pulses proportional to the flow of water. The sensor uses a Hall-effect mechanism to detect the rotation of an internal turbine.

3.4.2. Working Principle

The sensor consists of:

- A rotor (impeller) inside the water path
- A magnet attached to the rotor
- A Hall-effect sensor outside the chamber

Operation:

1. Water flows through the sensor → rotates the impeller
2. Magnet passes near Hall sensor → generates pulse
3. Each rotation produces a digital square wave pulse
4. Pulse frequency $\propto$ water flow rate

This allows non-contact measurement (sensor remains dry)



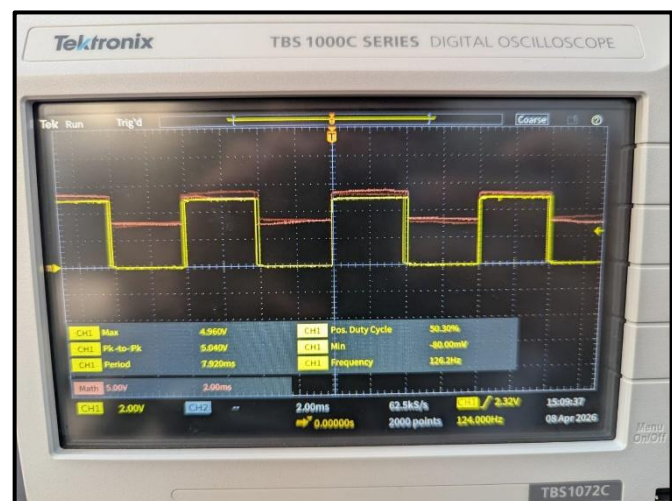
3.4.3. Technical Specification

Parameter	Value
Sensor Type	Hall-effect flow sensor
Output	Digital pulse (square wave)
Operating Voltage	5V
Flow Range	0.3 – 6 L/min
Pulse Output	~50% duty cycle
Accuracy	±5%
Pulse Relation	$F = 98 \times Q$
Pulses per Liter	5880 pulses

3.4.4. Oscilloscope Analysis:

Observed Parameters:

- Maximum Voltage ≈ 5.04 V
- Peak-to-Peak ≈ 5.12 V
- Frequency ≈ 126 Hz
- Period ≈ 7.92 ms
- Duty Cycle $\approx 50.1\%$



Interpretation:

- Clean square wave confirms correct sensor operation
- 0–5V signal indicates proper digital output
- ~50% duty cycle matches datasheet specification
- Frequency corresponds to flow rate

3.4.5. Flow Rate Calculation

From datasheet:

$$F = 98 \times Q$$

$$Q = \frac{F}{98}$$

Using measured frequency:

$$Q = \frac{126.2}{98} \approx 1.29 \text{ L/min}$$

Calculated Flow Rate $\approx 1.29 \text{ L/min}$

3.4.5. Implementation of Water Flow Sensor in System

The YF-S401 water flow sensor is interfaced with the Raspberry Pi Pico W using a digital input pin (GP13). The sensor produces pulse signals corresponding to the rotation of an internal turbine, which are counted using an interrupt-based method.

Hardware Interface

- Sensor output (yellow wire) → **GPIO GP13**
- Mode → **Interrupt (rising edge detection)**

```
flow_pin = Pin(13, Pin.IN)
```

```
flow_pin.irq(trigger=Pin.IRQ_RISING,
handler=count_pulse)
```

Each pulse corresponds to turbine rotation.

Pulse Counting Mechanism

- Every rising edge → interrupt triggered
- Pulse count incremented

```
def count_pulse(pin):
    global pulse_count, total_pulses
    pulse_count += 1
    total_pulses += 1
```

Flow Rate Measurement Method

The system measures flow over a **fixed time window (500 ms)**:

```
pulse_count = 0
time.sleep_ms(500)
```

Then converts it into flow rate:

$$\text{flow_rate} = (\text{pulse_count} * 120) / \text{PULSES_PER_LITER}$$

Explanation of Formula

From datasheet:

- 1 liter = **5880 pulses**

Since measurement is for 0.5 seconds:

- Multiply by **120** to scale to per minute:

$$\text{Flow Rate} = \frac{\text{pulse count} \times 120}{5880}$$

Total Water Measurement:

Total volume is calculated using cumulative pulses:

$$\text{total_liters} = \text{total_pulses} / \text{PULSES_PER_LITER}$$

This gives **total water used (in liters)**

System Integration

The flow sensor is used in:

1. LCD Display

- Shows real-time flow rate
- Shows total water usage

Pump Monitoring Logic

The system uses flow data to detect faults:

If pump ON and flow = 0 → fault (dry run / blockage)

Prevents damage to pump

Algorithm for integration of flow sensor:

1. Initialize flow sensor on GPIO pin (GP13)
2. Enable interrupt on rising edge
3. On each pulse:
 increment pulse_count and total_pulses
4. Every 500 ms:
 reset pulse_count
 wait 500 ms
 calculate flow_rate:

$$\text{flow_rate} = (\text{pulse_count} \times 120) / 5880$$
5. Calculate total water:

$$\text{total_liters} = \text{total_pulses} / 5880$$
6. Display values on LCD
7. Send values to dashboard
8. Use flow data for pump fault detection

3.5. Capacitive Soil Moisture Sensor

3.5.1. Introduction

The capacitive soil moisture sensor (V2.0) is used to measure the moisture content of soil in the irrigation system. Unlike resistive sensors, it operates on the principle of capacitance variation, which makes it resistant to corrosion and suitable for long-term deployment. The sensor provides an analog voltage output proportional to soil moisture, which is read using the ADC of the Raspberry Pi Pico W.

3.5.2. Working Principle

The sensor works based on capacitive sensing:

- The sensor contains two conductive plates embedded in PCB
- Soil acts as the dielectric medium
- Moisture in soil changes dielectric constant:
 - Dry soil → low capacitance
 - Wet soil → high capacitance

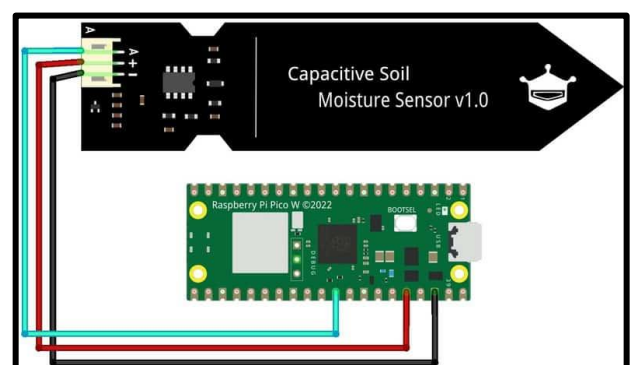
This change in capacitance is converted into an analog voltage output

Key Behavior:

- Dry soil → higher voltage → higher ADC value
- Wet soil → lower voltage → lower ADC value

3.5.3. Technical Specifications:

Parameter	Value
Sensor Type	Capacitive
Operating Voltage	3.3V – 5.5V
Output	Analog (0 – 3V)
Interface	3-pin (VCC, GND, Analog)
Dimensions	98 × 23 × 4 mm
Application	Soil moisture detection



3.5.4. Sensor Calibration:

The sensor was calibrated using real experimental values:

Condition	Raw ADC Value	Moisture (%)
Air (Dry)	~64500	0%
Fully Wet	~34100	100%
Disconnected	~65535	Error

Conversion Formula

Since the sensor is **inverted (higher = drier)**:

$$\text{Moisture} = \frac{(SOIL_DRY - value)}{(SOIL_DRY - SOIL_WET)} \times 100$$

From Code:

```
soil_percent = ((64500 - val) / (64500 - 34100)) * 100
```

Signal Processing (Noise Reduction)

To improve stability, a **trimmed mean filter** is used:

Take 10 readings

Remove top 20% and bottom 20%

Average remaining values

This reduces:

- Noise
- Power fluctuations
- ADC instability

Implementation in System

The sensor is connected to:

- **GPIO26 (ADC0)** of Raspberry Pi Pico

Processing Steps:

1. Read analog value using ADC
2. Apply filtering (10 samples)
3. Convert raw value → moisture %
4. Determine soil condition

Soil Status Classification:

Moisture (%)	Status
< 30%	DRY
30–60%	OK
> 60%	WET
Invalid	ERR

Pump Control Logic (Based on Soil):

IF soil < 40% → Pump ON

IF soil ≥ 60% → Pump OFF

Additional Safety:

- If sensor disconnected → Pump OFF
- Combined with rain + tank logic

3.6. 20×4 LCD Display:

The 20×4 LCD display is used in the system to provide real-time visualization of sensor data such as soil moisture, temperature, humidity, water level, and flow rate. It can display 20 characters per line across 4 lines, allowing multiple parameters to be shown simultaneously.

The display is interfaced with the Raspberry Pi Pico using the I²C communication protocol, which reduces the number of required GPIO pins. The LCD controller converts incoming data into visible characters on the screen.

In this system, the display is used to show:

- Soil moisture percentage
- Tank water level
- Temperature and humidity
- Pump status and flow rate

The LCD updates periodically, providing the user with instant feedback about the irrigation system status.



3.7. 12V Pump

The 12V DC diaphragm pump is used to supply water in the irrigation system. It operates using a motor-driven diaphragm mechanism that creates suction and pushes water through the outlet. This type of pump is **self-priming**, meaning it can draw water without manual priming.

The pump runs on an external 12V supply and is controlled using a relay module, as the Raspberry Pi Pico cannot directly drive high-current loads.



Key Features

- Operates at **12V DC**
- **Self-priming** (can suck water automatically)
- Provides steady pressure for irrigation
- Suitable for small-scale agricultural systems

Working in System

- Pump turns **ON** when soil moisture is low
- Pump turns **OFF** when moisture is sufficient or rain is detected
- Controlled via **relay module**

Protection

- A **flyback diode** is connected across the pump
- Protects circuit from **back EMF voltage spikes**

3.8 Raspberry Pi Pico W (Microcontroller Unit):

The Raspberry Pi Pico W is the main controller used in the smart irrigation system. It is based on the RP2040 microcontroller and features built-in Wi-Fi connectivity, enabling both control and remote monitoring.

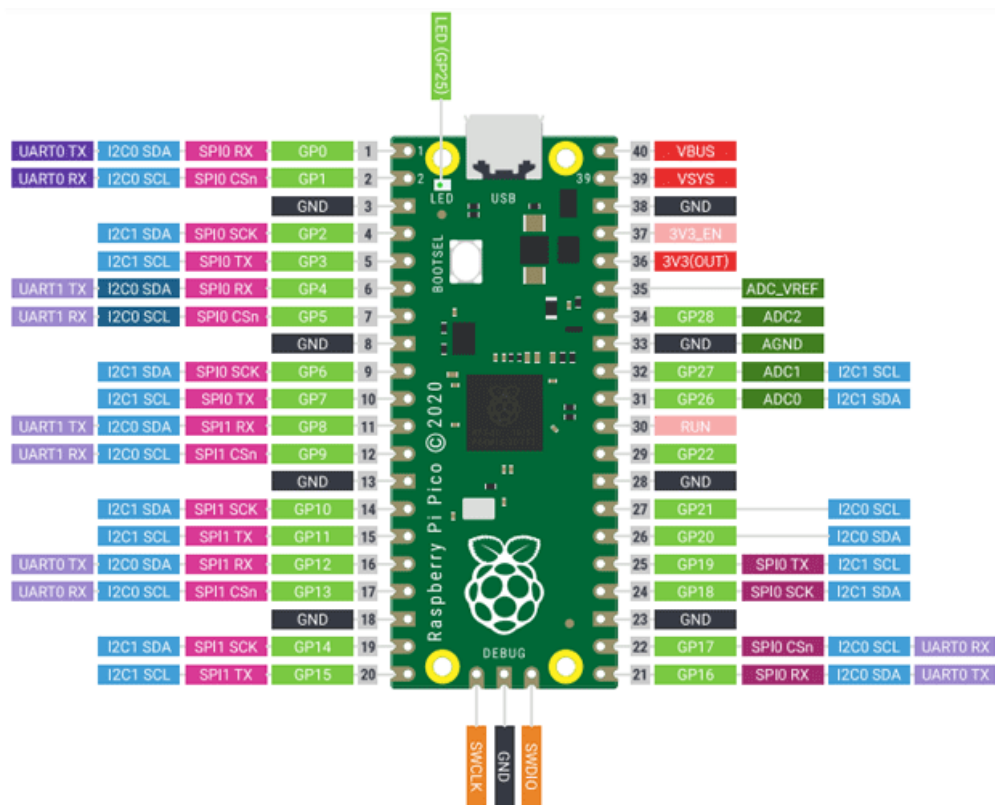
Key Features

- Dual-core ARM Cortex-M0+ processor
- Operating voltage: 3.3V
- Multiple GPIO pins (digital & analog)
- 12-bit ADC for sensor readings
- Built-in Wi-Fi module (CYW43439)
- Low power consumption

Role in System

The Pico W acts as the brain of the system:

- Reads data from all sensors (soil, rain, tank, flow, DHT11)
- Processes and calibrates sensor values
- Controls the relay to operate the pump
- Displays data on LCD
- Hosts a web server for remote monitoring



4. Control System using Raspberry Pi Pico W:

4.1. Introduction

The Raspberry Pi Pico W serves as the central controller of the smart irrigation system. It is responsible for acquiring data from multiple sensors, processing the data, making control decisions, and actuating the water pump through a relay. Additionally, it provides wireless connectivity for remote monitoring via a web dashboard.

4.2. System Overview

The control system integrates the following components:

- Soil Moisture Sensor (ADC input)
- Water Level Sensor (ADC input)
- Rainfall Sensor (Digital input)
- Flow Sensor (Interrupt-based input)
- DHT11 Temperature & Humidity Sensor (Digital communication)
- Relay Module (Digital output)
- LCD Display (I²C communication)
- Wi-Fi Module (built-in Pico W)

4.3. Data Acquisition

The Pico W continuously reads sensor data using different interfaces:

Analog Inputs (ADC)

- Soil moisture and water level sensors
- Converted using internal ADC (scaled to 16-bit)

Digital Inputs

- Rain sensor → HIGH/LOW signal
- DHT11 → single-wire protocol

Interrupt Input

- Flow sensor → pulse counting using GPIO interrupt

4.4. Data Processing

The collected sensor data is processed as follows:

- **Filtering:** Multiple ADC readings are averaged to reduce noise
- **Calibration:** Raw values converted into meaningful units
- **Validation:** Sensor faults (e.g., disconnected sensor) are detected

Control Logic

The control system uses rule-based logic to determine pump operation.

Primary Conditions:

IF soil moisture < threshold → irrigation required

IF rain detected → irrigation stopped

IF tank level low → pump disabled

Final Decision Logic:

IF (soil is dry) AND (no rain) AND (tank has water)

→ Pump ON

ELSE

→ Pump OFF

Hysteresis Control

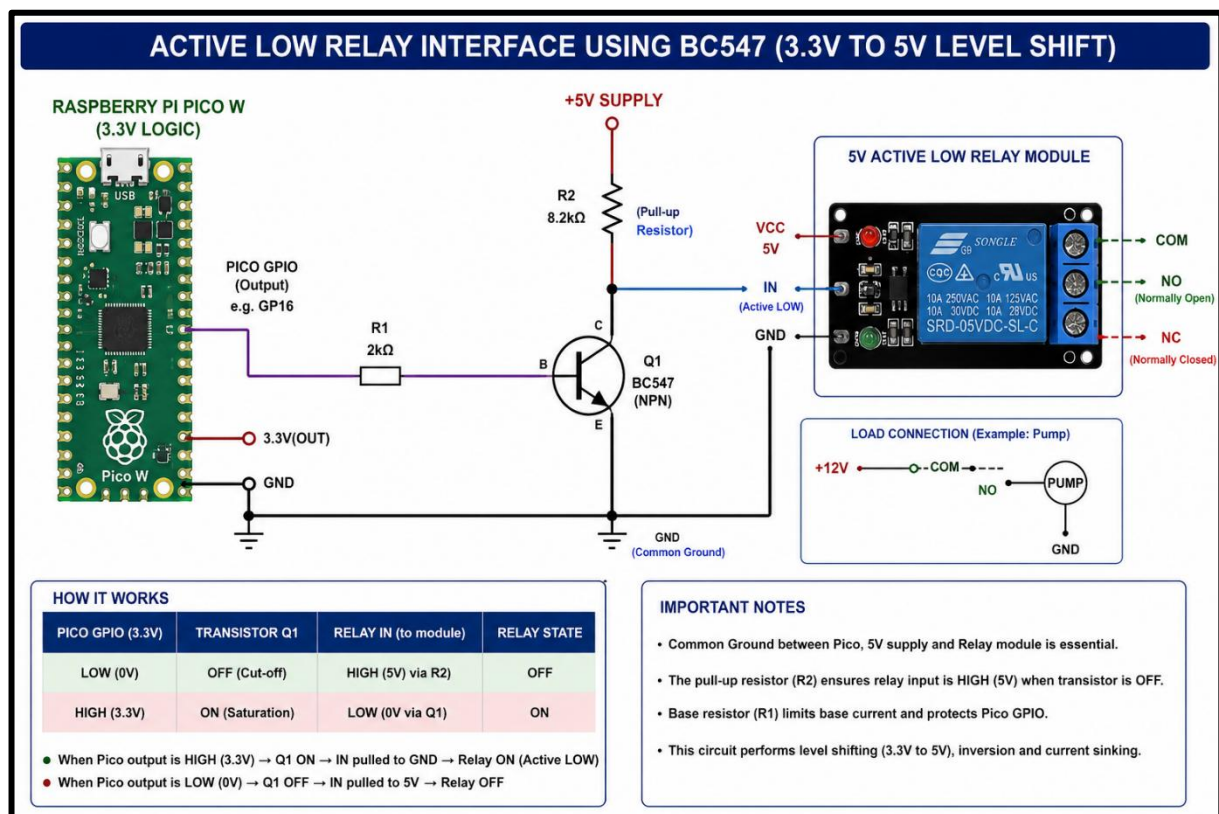
To avoid frequent ON/OFF switching:

- Pump ON at **low moisture threshold (~40%)**
- Pump OFF at **high moisture threshold (~60%)**

5. Actuation System (Relay Control using BC547 Level Shifting)

5.1. Introduction

The actuation system is responsible for controlling the 12V water pump based on decisions made by the Raspberry Pi Pico W. Since the Pico operates at 3.3V logic and the relay module requires a 5V control signal (active LOW), a transistor-based level shifting circuit using a BC547 NPN transistor is implemented.



Need for Level Shifting

- Raspberry Pi Pico GPIO output: 3.3V
- Relay module input requirement: 5V logic (active LOW)

Direct connection is unreliable because:

- 3.3V may not reliably drive relay module
- Logic mismatch can cause false triggering

5.2. Circuit Components

Component	Function
BC547 Transistor	Acts as switch & level shifter
R1 (2kΩ)	Limits base current
R2 (8.2kΩ)	Pull-up resistor to 5V
Relay Module	Controls pump
12V Pump	Load
Common Ground	Reference for all components

5.3. Working Principle

The circuit performs three functions simultaneously:

1. Level shifting ($3.3V \rightarrow 5V$)
2. Signal inversion
3. Current amplification

Case 1: Pico Output LOW (0V)

- No base current $\rightarrow$ transistor OFF
- Collector pulled HIGH via R2 (5V)
- Relay input = HIGH
- Relay OFF

Case 2: Pico Output HIGH (3.3V)

- Base current flows through R1
- Transistor turns ON (saturation)
- Collector pulled to GND
- Relay input = LOW
- Relay ON

5.4 Truth Table:

Pico Output	Transistor	Relay Input	Relay State	Pump
LOW (0V)	OFF	HIGH (5V)	OFF	OFF
HIGH (3.3V)	ON	LOW (0V)	ON	ON

5.5. Relay Operation

The relay module is **active LOW**, meaning:

- LOW signal → Relay ON
- HIGH signal → Relay OFF

The relay switches the **12V supply** to the pump:

- COM → +12V
- NO → Pump
- Pump → GND

Role in System

The actuation system converts digital decisions into physical action:

If irrigation required → Relay ON → Pump ON

If irrigation not required → Relay OFF → Pump OFF

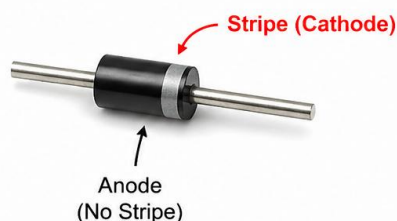
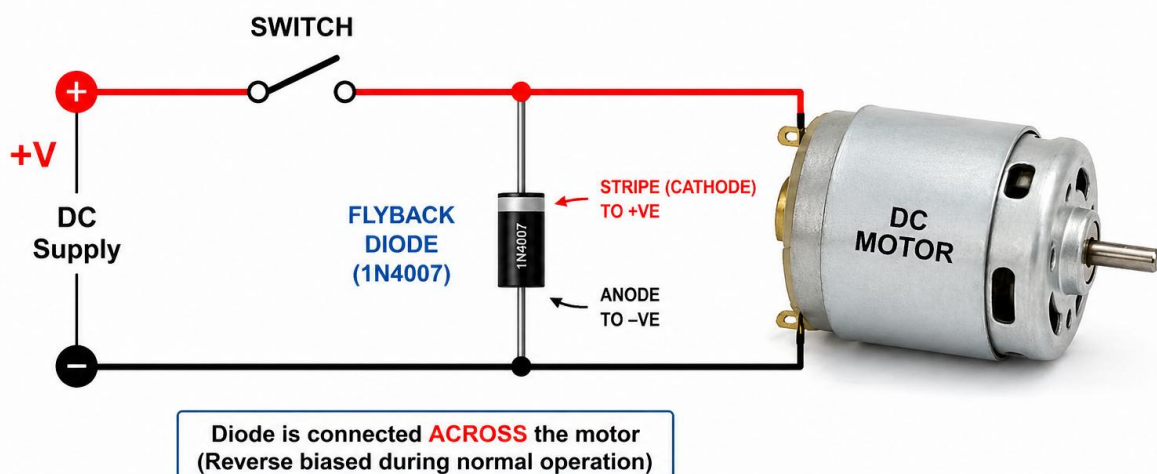
6. Flyback Diode Protection

A flyback diode is connected across the 12V pump to protect the circuit from voltage spikes generated when the pump is switched OFF. Since the pump is an inductive load, it produces a high back EMF when current is interrupted.

The diode is connected in reverse bias:

- Cathode $\rightarrow$ +12V
- Anode $\rightarrow$ Ground

During normal operation, the diode does not conduct. When the pump is turned OFF, the diode provides a path for the induced current, preventing damage to the relay and control circuitry.



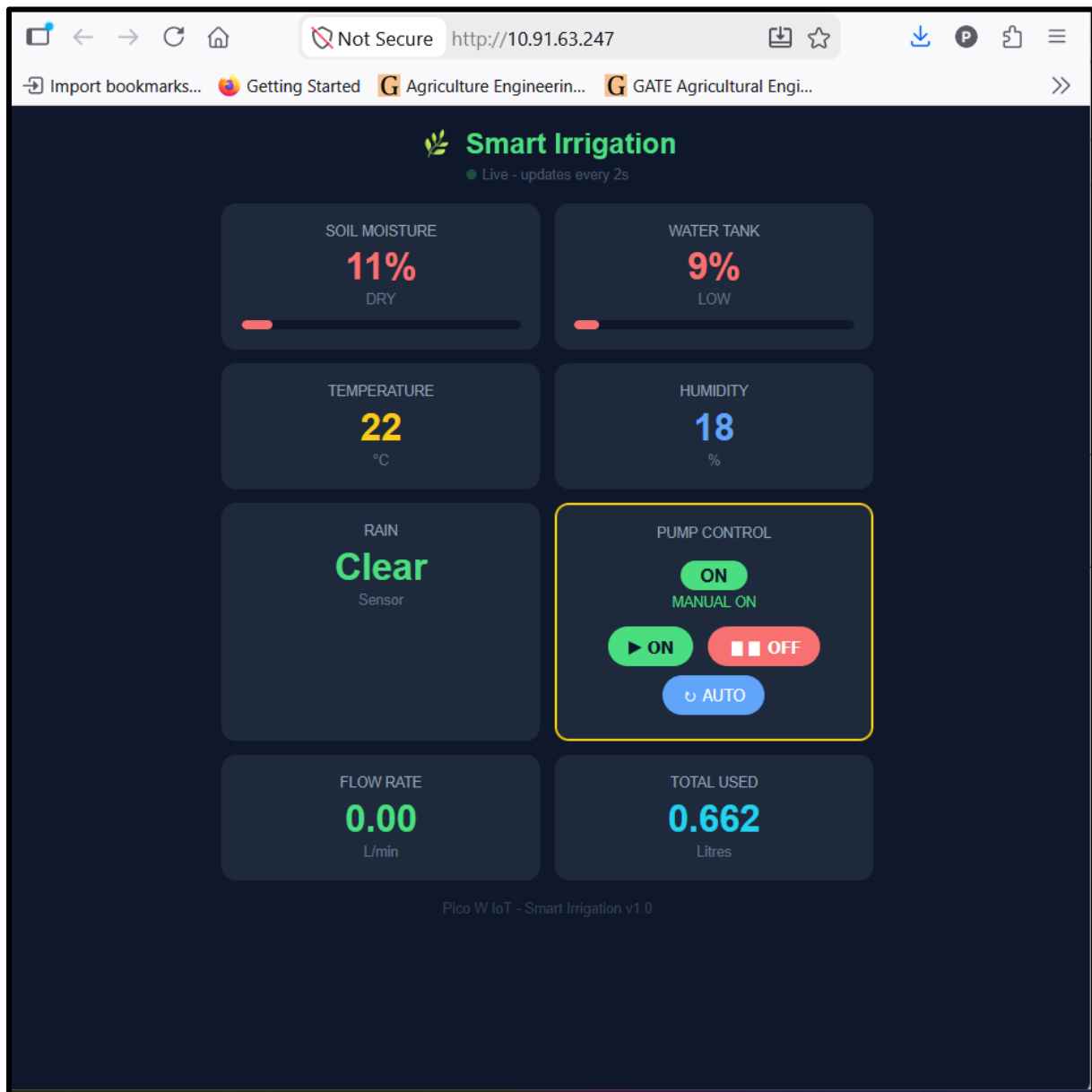
HOW IT WORKS

When switch is **ON**: current flows through the motor.

When switch is **OFF**: the collapsing magnetic field in the motor creates a high voltage spike (back EMF).

The diode provides a path for this current, protecting the switch/transistor from damage.

7. Web Dashboard Interface



The Raspberry Pi Pico W hosts a web-based dashboard that allows real-time monitoring and control of the irrigation system through a browser using its IP address.

The dashboard displays key parameters such as soil moisture, tank level, temperature, humidity, rain status, flow rate, and total water usage. The data is updated periodically (every ~2 seconds) using a JSON API served by the Pico.

Pump Control

The dashboard provides three control modes:

- **Manual ON** → User turns pump ON

- **Manual OFF** → User turns pump OFF
- **AUTO Mode** → Pump controlled automatically based on sensor conditions

Code Used:

```
from machine import ADC, Pin
import dht
import time
import utime
import network
import socket
import json
import gc

# =====
#  WIFI CREDENTIALS
#  =====
WIFI_SSID      = "Parthibg60"
WIFI_PASSWORD = "Parthib123"

# =====
#  LCD SETUP (4-bit mode, 20x4)
#  RS=GP17  E=GP18  D4=GP19  D5=GP20  D6=GP21  D7=GP22
#  =====
RS = Pin(17, Pin.OUT)
E  = Pin(18, Pin.OUT)
D4 = Pin(19, Pin.OUT)
D5 = Pin(20, Pin.OUT)
D6 = Pin(21, Pin.OUT)
D7 = Pin(22, Pin.OUT)
```

```

def pulse_enable():
    E.value(0); utime.sleep_us(1)
    E.value(1); utime.sleep_us(1)
    E.value(0); utime.sleep_us(50)

def send_nibble(nibble):
    D4.value((nibble >> 0) & 1)
    D5.value((nibble >> 1) & 1)
    D6.value((nibble >> 2) & 1)
    D7.value((nibble >> 3) & 1)
    pulse_enable()

def send_byte(byte, mode):
    RS.value(mode)
    send_nibble(byte >> 4)
    send_nibble(byte & 0x0F)
    utime.sleep_us(50)

def lcd_init():
    utime.sleep_ms(50)
    RS.value(0)
    send_nibble(0x03); utime.sleep_ms(5)
    send_nibble(0x03); utime.sleep_ms(1)
    send_nibble(0x03); utime.sleep_us(200)
    send_nibble(0x02)
    send_byte(0x28, 0)    # 4-bit, 2-line, 5x8
    send_byte(0x0C, 0)    # display on, cursor off
    send_byte(0x06, 0)    # entry mode: increment
    send_byte(0x01, 0)    # clear

```

```

        utime.sleep_ms(2)

def lcd_clear():
    send_byte(0x01, 0)
    utime.sleep_ms(2)

def lcd_set_cursor(row, col):
    row_offsets = [0x00, 0x40, 0x14, 0x54]
    send_byte(0x80 | (row_offsets[row] + col), 0)

def lcd_print(text):
    for char in text:
        send_byte(ord(char), 1)

def ljust(text, width):
    while len(text) < width:
        text += " "
    return text[:width]

def lcd_print_row(row, text):
    lcd_set_cursor(row, 0)
    lcd_print(ljust(text, 20))

# =====
# SENSORS
# Soil moisture (capacitive) = GP26 (ADC0)
# Water level                = GP27 (ADC1)
# DHT11 temp/humidity        = GP15
# Rain sensor (digital)      = GP14
# Flow sensor (interrupt)    = GP13

```

```

# =====
soil_sensor = ADC(Pin(26))
water_sensor = ADC(Pin(27))
dht_sensor = dht.DHT11(Pin(15))
rain_sensor = Pin(14, Pin.IN)
flow_pin = Pin(13, Pin.IN)

# =====
# RELAY (pump control) GP16
# value=0 at boot -> pump starts OFF (safe default)
# =====
relay = Pin(16, Pin.OUT, value=0)
pump_on = False
pump_mode = 2 # 0>manual OFF 1>manual ON 2=AUTO

# =====
# FLOW SENSOR (YF-S401)
# Datasheet: 1 litre = 5880 pulses
# Formula:  $F = 98 * Q$  (Q in L/min)
# =====
pulse_count = 0
total_pulses = 0
PULSES_PER_LITER = 5880

def count_pulse(pin):
    global pulse_count, total_pulses
    pulse_count += 1
    total_pulses += 1

flow_pin.irq(trigger=Pin.IRQ_RISING, handler=count_pulse)

```

```

# =====
# CALIBRATION (all values from real sensor measurements)
# =====

# -- Capacitive Soil Moisture Sensor (GPIO26) -----
# Higher raw value = drier | Lower raw value = wetter
# Air reading ~64500 = 0% moisture -> pump turns ON (correct)
# SOIL_DISCONNECTED raised to 65200 so air reading is NOT
# treated as disconnected (ADC max = 65535 when truly unplugged)
SOIL_DRY          = 64500    # bare in air    -> 0%  moisture
SOIL_WET          = 34100    # fully in water -> 100% moisture
SOIL_DISCONNECTED = 65200    # truly unplugged -> ADC reads
~65535

# -- Water Level Sensor (GPIO27) -----
# Measured values:
#   Dry / no water -> ~1150
#   30% submerged -> ~19800
#   100% submerged -> ~29000
WATER_EMPTY      = 1150     # dry / no water -> 0%
WATER_FULL       = 29000    # fully submerged -> 100%
WATER_NOISE_FLOOR = 800     # below this      -> disconnected /
error
# 20% raw threshold = (29000 - 1150) * 0.20 + 1150 = 6720
WATER_LOW_RAW    = 6720     # below this      -> tank LOW ->
stop pump

# -- Pump Hysteresis Thresholds -----
PUMP_ON_SOIL     = 40       # soil < 40% -> turn pump ON
PUMP_OFF_SOIL    = 60       # soil >= 60% -> turn pump OFF

```

```
WATER_TANK_MIN    = 20          # tank <= 20% -> stop pump (protect
pump)
```

```
# =====
```

```
#  SHARED SENSOR DATA  (served as JSON via /data endpoint)
```

```
# =====
```

```
sensor_data = {
    "soil"          : 0,
    "soil_raw"      : 0,
    "soil_status": "---",
    "tank"          : 0,
    "tank_raw"      : 0,
    "tank_status": "---",
    "temp"          : 0,
    "hum"           : 0,
    "rain"          : False,
    "pump"          : False,
    "flow"          : 0.0,
    "total"         : 0.0,
    "pump_mode"     : 2
}
```

```
# =====
```

```
#  STABLE ADC READER
```

```
#  Takes 10 samples, discards top & bottom 20%, returns mean
```

```
#  Reduces noise from power supply / other sensors
```

```
# =====
```

```
def read_adc_stable(sensor, samples=10):
```

```
    readings = []
```

```
    for _ in range(samples):
```

```

        sensor.read_u16()          # discard first (charge
settling)
        time.sleep_ms(5)
        readings.append(sensor.read_u16())
    readings.sort()
    trim    = samples // 5          # trim 2 from each end
    trimmed = readings[trim:-trim]
    return sum(trimmed) // len(trimmed)

# =====
#  CONVERSION FUNCTIONS
# =====

def soil_percent(val):
    """
    Capacitive sensor: higher raw = drier
    Formula is inverted compared to standard ADC percent.
    Returns 0 (dry/air) to 100 (wet).
    Air reading ~64500 -> 0% -> pump turns ON correctly.
    """
    if SOIL_DRY == SOIL_WET:
        return 0
    return max(0, min(100, int(((SOIL_DRY - val) / (SOIL_DRY -
SOIL_WET)) * 100)))

def water_percent(val):
    """
    Returns 0-100% based on raw ADC value.
    Returns -1 if reading is below noise floor
    (error/disconnected).
    Simple linear map: WATER_EMPTY=0%, WATER_FULL=100%
    """

```

```

    if val < WATER_NOISE_FLOOR:
        return -1
    if WATER_FULL == WATER_EMPTY:
        return 0
    return max(0, min(100, int(((val - WATER_EMPTY) /
(WATER_FULL - WATER_EMPTY)) * 100)))

# =====
# DHT11 (temperature & humidity)
# Retries up to 3 times on failure, keeps last good reading
# =====

temp = 27
hum  = 40

def read_dht():
    global temp, hum
    for _ in range(3):
        try:
            time.sleep_ms(200)
            dht_sensor.measure()
            temp = dht_sensor.temperature()
            hum  = dht_sensor.humidity()
            return temp, hum
        except:
            time.sleep_ms(100)
    return temp, hum    # return last good reading on failure

# =====
# PUMP CONTROL LOGIC
#
# AUTO mode rules:

```

```

# ON when: soil < 40% AND no rain AND tank > 20%
# OFF when: soil >= 60% OR rain OR tank <= 20%
#           OR soil truly unplugged (raw > 65200)
#           OR water sensor noise/error
#
# NOTE: Air/dry soil reads ~64500 = 0% moisture
#       This correctly triggers pump ON in AUTO mode.
# Hysteresis (40% ON / 60% OFF) prevents rapid cycling.
# Tank LOW uses raw threshold (WATER_LOW_RAW) for reliability.
# =====
def control_pump(soil_moist, water_level, rain_detected,
soil_raw, water_raw):
    global pump_on, pump_mode

    # Manual override modes
    if pump_mode == 1:
        relay.value(1); pump_on = True; return
    if pump_mode == 0:
        relay.value(0); pump_on = False; return

    # AUTO mode safety checks
    if soil_raw > SOIL_DISCONNECTED:      # truly unplugged
(~65535)
        relay.value(0); pump_on = False; return
    if water_raw < WATER_NOISE_FLOOR:    # water sensor error
        relay.value(0); pump_on = False; return
    if water_raw < WATER_LOW_RAW:        # tank below 20% ->
protect pump
        relay.value(0); pump_on = False; return

    # AUTO mode pump ON condition

```

```

    if (soil_moist < PUMP_ON_SOIL
        and not rain_detected
        and water_level > WATER_TANK_MIN):
        relay.value(1); pump_on = True

# AUTO mode pump OFF condition
elif (soil_moist >= PUMP_OFF_SOIL
        or rain_detected
        or water_level <= WATER_TANK_MIN):
        relay.value(0); pump_on = False

# Between 40-60% with no special condition -> hold current
state

# =====
#  WIFI
# =====

wlan = network.WLAN(network.STA_IF)
ip    = None

def connect_wifi():
    global ip
    wlan.active(True)
    if wlan.isconnected():
        ip = wlan.ifconfig()[0]
        return ip
    wlan.connect(WIFI_SSID, WIFI_PASSWORD)
    lcd_print_row(2, "Connecting WiFi... ")
    for i in range(20):
        if wlan.isconnected():
            ip = wlan.ifconfig()[0]

```

```

        print("Connected! IP:", ip)
        lcd_print_row(2, ljust("IP:" + ip, 20))
        return ip
    time.sleep(1)
    lcd_print_row(3, ljust("Attempt " + str(i + 1) + "/20",
20))
    print("Connecting... attempt", i + 1)
    print("WiFi failed!")
    lcd_print_row(2, "WiFi FAILED!          ")
    return None

def check_wifi():
    global ip
    if not wlan.isconnected():
        print("WiFi lost! Reconnecting...")
        ip = connect_wifi()
    else:
        ip = wlan.ifconfig()[0]
    return ip

# =====
# HTTP FILE SERVER (serves index.html for dashboard)
# =====

def send_file(conn, filename):
    try:
        conn.sendall(
            "HTTP/1.1 200 OK\r\n"
            "Content-Type: text/html\r\n"
            "Connection: close\r\n\r\n"
        )
        with open(filename, 'r') as f:

```

```

        while True:
            chunk = f.read(256)
            if not chunk:
                break
            conn.sendall(chunk.encode('utf-8'))
            time.sleep_ms(20)
        gc.collect()
    except Exception as e:
        print("Send file error:", e)

# =====
# HTTP REQUEST HANDLER
#
# GET /          -> serves index.html (dashboard)
# GET /data      -> returns JSON sensor data
# GET /pump?state=0 -> manual OFF
# GET /pump?state=1 -> manual ON
# GET /pump?state=2 -> AUTO mode
# =====

def handle_request(conn):
    global pump_mode
    try:
        conn.settimeout(5.0)
        request = b""
        while True:
            try:
                part = conn.recv(256)
                if not part:
                    break
                request += part

```

```

        if b"\r\n\r\n" in request:
            break
    except:
        break

request = request.decode('utf-8')
print("Request:", request[:60])

# -- Pump mode control --
if "/pump?state=" in request:
    try:
        state =
int(request.split("/pump?state=")[1][0])
        pump_mode = state
        sensor_data["pump_mode"] = pump_mode
        print("Pump mode set to:", pump_mode)
    except:
        pass
    resp = (
        "HTTP/1.1 200 OK\r\n"
        "Content-Type: application/json\r\n"
        "Access-Control-Allow-Origin: *\r\n"
        "Connection: close\r\n\r\n"
        + json.dumps({"pump_mode": pump_mode})
    )
    conn.sendall(resp.encode('utf-8'))

# -- JSON sensor data --
elif "/data" in request:
    sensor_data["pump_mode"] = pump_mode
    body = json.dumps(sensor_data)

```

```

        resp = (
            "HTTP/1.1 200 OK\r\n"
            "Content-Type: application/json\r\n"
            "Access-Control-Allow-Origin: *\r\n"
            "Connection: close\r\n\r\n"
            + body
        )
        conn.sendall(resp.encode('utf-8'))

# -- Dashboard HTML --
else:
    send_file(conn, "index.html")

except Exception as e:
    print("Request error:", e)
finally:
    conn.close()
    gc.collect()

# =====
#  STARTUP SEQUENCE
#  =====

lcd_init()
lcd_clear()
lcd_print_row(0, "  Smart Irrigation  ")
lcd_print_row(1, "      System v1.0    ")
lcd_print_row(2, "  Initializing...   ")
lcd_print_row(3, "  Please wait...    ")
time.sleep(2)

```

```

gc.collect()
ip = connect_wifi()
time.sleep(1)
lcd_clear()

server = None
if ip:
    addr = socket.getaddrinfo("0.0.0.0", 80)[0][-1]
    server = socket.socket()
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind(addr)
    server.listen(2)
    server.setblocking(False)
    print("Dashboard : http://", ip)
    print("JSON API : http://", ip, "/data")

# =====
#  MAIN LOOP
# =====
dht_counter = 0    # read DHT every 5 iterations (~5s)
wifi_counter = 0   # check WiFi every 30 iterations (~30s)
display_page = 0   # LCD page (0 or 1)
page_counter = 0   # rotate page every 5 iterations

while True:

    # -- WiFi watchdog --
    wifi_counter += 1
    if wifi_counter >= 30:
        wifi_counter = 0

```

```
    check_wifi()

# -- Read ADC sensors --
soil_raw = read_adc_stable(soil_sensor)
water_raw = read_adc_stable(water_sensor)

# -- Read DHT11 every 5 loops (sensor needs ~1s between
reads) --
dht_counter += 1
if dht_counter >= 5:
    temp, hum = read_dht()
    dht_counter = 0

# -- Rain sensor (LOW = rain detected) --
rain_detected = rain_sensor.value() == 0

# -- Flow rate (count pulses over 500ms window) --
pulse_count = 0
time.sleep_ms(500)
# pulse_count x 120 scales 500ms window to per-minute
# divide by 5880 (pulses/litre) -> L/min
flow_rate = (pulse_count * 120) / PULSES_PER_LITER
total_liters = total_pulses / PULSES_PER_LITER

# -- Convert raw ADC to percentages --
soil_moist = soil_percent(soil_raw)
water_level = water_percent(water_raw)

# -- Pump control --
control_pump(soil_moist, water_level, rain_detected,
soil_raw, water_raw)
```

```
# -- Soil status label --
if soil_raw > SOIL_DISCONNECTED:
    soil_status = "ERR"    # truly unplugged
elif soil_moist < 30:
    soil_status = "DRY"    # includes air reading (0%)
elif soil_moist < 60:
    soil_status = "OK "
else:
    soil_status = "WET"

# -- Tank status label (uses raw value for LOW check) --
wl = water_level if water_level >= 0 else 0
if water_raw < WATER_NOISE_FLOOR:
    tank_status = "ERR"
elif water_raw < WATER_LOW_RAW:    # raw below 20%
threshold
    tank_status = "LOW"
elif wl < 60:
    tank_status = "MED"
else:
    tank_status = "OK "

# -- Update shared data dict --
sensor_data["soil"]      = soil_moist
sensor_data["soil_raw"]  = soil_raw
sensor_data["soil_status"] = soil_status
sensor_data["tank"]      = wl
sensor_data["tank_raw"]  = water_raw
sensor_data["tank_status"] = tank_status
sensor_data["temp"]      = temp
```

```

    sensor_data["hum"]          = hum
    sensor_data["rain"]         = rain_detected
    sensor_data["pump"]         = pump_on
    sensor_data["flow"]         = flow_rate
    sensor_data["total"]        = total_liters
    sensor_data["pump_mode"]    = pump_mode

    # -- LCD page rotation (every 5 loops ~5s) --
    page_counter += 1
    if page_counter >= 5:
        page_counter = 0
        display_page = (display_page + 1) % 2

    if display_page == 0:
        # Page 0: live sensor readings
        lcd_print_row(0, "T:{:2d}C H:{:2d}% {}".format(
            temp, hum, "RAIN" if rain_detected else "    "))
        lcd_print_row(1, "Soil:{:3d}% {}".format(soil_moist,
soil_status))
        lcd_print_row(2, "Tank:{:3d}% {}".format(wl,
tank_status))
        lcd_print_row(3, "Pump:{}".format(
            "ON " if pump_on else "OFF", flow_rate))
    else:
        # Page 1: flow totals, IP, pump mode
        lcd_print_row(0, "Total: {:.3f} L".format(total_liters))
        lcd_print_row(1, "Flow : {:.3f} L/m".format(flow_rate))
        lcd_print_row(2, ljust("IP:" + (ip if ip else "NoWiFi"),
20))
        lcd_print_row(3, "Mode:{}".format(
            "MAN-ON " if pump_mode == 1 else

```

```

        "MAN-OFF" if pump_mode == 0 else
        "AUTO  ")

# -- Serial debug output --
print("=" * 36)
print("Soil:{:3d}%({}) Tank:{:3d}%({})".format(
    soil_moist, soil_status, wl, tank_status))
print("Temp: {}C Hum: {}% Rain: {} Pump: {}".format(
    temp, hum,
    "YES" if rain_detected else "NO ",
    "ON " if pump_on else "OFF"))
print("Mode: {} Flow: {:.3f}L/m Total: {:.4f}L".format(
    pump_mode, flow_rate, total_liters))
print("RawSoil: {} RawTank: {}".format(soil_raw, water_raw))
print("=" * 36)

# -- Handle one HTTP request per loop --
if server:
    try:
        conn, addr = server.accept()
        print("Client:", addr)
        handle_request(conn)
    except OSError:
        pass

gc.collect()

```